

Implementation and Analysis of a Third-Order Newton-Type Method for Nonlinear Systems

Carlota Arzúa Alonso

Abstract

This project implements and analyzes a third-order Newton-type method for solving nonlinear systems of equations. The method is based on the modified Newton iteration introduced by Darvishi and Barati, which improves the classical Newton method by using an intermediate Newton step and an additional function evaluation. The algorithm was implemented in MATLAB and tested on two problems: a nonlinear optimization problem in three variables and an exponential regression problem in two variables. The project demonstrates skills in numerical methods, MATLAB programming, Jacobian construction, iterative algorithms, and convergence analysis.

1 Project Overview

The purpose of this project was to take a mathematical algorithm from numerical analysis and turn it into a working MATLAB implementation. The algorithm solves nonlinear systems of the form

$$F(x) = 0, \tag{1}$$

where F is a vector-valued nonlinear function. These types of systems appear in optimization, regression, engineering models, data fitting, and scientific computing.

The main focus of the project was not only to run the algorithm, but also to understand how it behaves under different problem settings. In particular, the project studies when the method moves toward a solution and when it becomes unstable because of the starting point or the shape of the nonlinear system.

From an industry perspective, this project shows the ability to:

- translate mathematical formulas into executable code;
- implement iterative algorithms in MATLAB;
- work with nonlinear systems and optimization problems;
- construct and use Jacobian matrices;
- analyze numerical results instead of only reporting outputs; and
- communicate technical findings clearly.

2 Technical Background

The classical Newton method is one of the most common algorithms for solving nonlinear equations and systems. For a nonlinear system $F(x) = 0$, the classical Newton update is

$$x_{n+1} = x_n - F'(x_n)^{-1}F(x_n), \quad (2)$$

where $F'(x_n)$ is the Jacobian matrix evaluated at the current approximation x_n .

The third-order Newton-type method used in this project modifies the classical method by first computing an intermediate point:

$$y_n = x_n - F'(x_n)^{-1}F(x_n). \quad (3)$$

Then, instead of only using $F(x_n)$, the method also evaluates the function at y_n and updates the estimate using

$$x_{n+1} = x_n - F'(x_n)^{-1} (F(x_n) + F(y_n)). \quad (4)$$

This extra evaluation can improve the convergence order of the method. When the initial guess is close enough to the solution and the problem behaves well, the method can converge faster than the classical Newton method. However, like other Newton-type methods, it is still sensitive to the initial guess and the structure of the nonlinear system.

3 Implementation Approach

The algorithm was implemented in MATLAB using a direct iterative structure. Each iteration follows the same general process:

1. Start with the current vector x_n .
2. Evaluate the nonlinear system $F(x_n)$.
3. Build the Jacobian matrix $F'(x_n)$.
4. Compute the intermediate Newton point y_n .
5. Evaluate $F(y_n)$.
6. Compute the next approximation x_{n+1} .
7. Repeat for a fixed number of iterations.

A simplified version of the algorithm is shown below.

Listing 1: Simplified MATLAB structure for the third-order Newton-type method

```
1 for iter = 1:NumIter
2     % Current values
3     x1 = xvect(1);
4     x2 = xvect(2);
5
6     % Evaluate F(x) and Jacobian F'(x)
7     Fvect_x = ...;
8     F_prime_x = ...;
```

```

9
10     % Intermediate Newton step
11     yvect = xvect - inv(F_prime_x) * Fvect_x;
12
13     % Evaluate F(y)
14     Fvect_y = ...;
15
16     % Modified Newton update
17     xvect = xvect - inv(F_prime_x) * (Fvect_x + Fvect_y);
18 end

```

The most important part of the implementation was building the correct Jacobian matrix. Since Newton-type methods rely directly on derivative information, even small mistakes in the Jacobian can change the behavior of the algorithm.

4 Experiment 1: Nonlinear Optimization System

The first problem involved minimizing the function

$$f(x, y, z) = (x - 1)^4 + (y - 3)^4 + (z - 5)^4 + (x - y)^6 + (y - z)^6 + (x - z)^6. \quad (5)$$

To use the Newton-type method, the optimization problem was converted into a system of nonlinear equations by taking the partial derivatives and setting them equal to zero:

$$\frac{\partial f}{\partial x} = 4(x - 1)^3 + 6(x - y)^5 + 6(x - z)^5 = 0, \quad (6)$$

$$\frac{\partial f}{\partial y} = 4(y - 3)^3 - 6(x - y)^5 + 6(y - z)^5 = 0, \quad (7)$$

$$\frac{\partial f}{\partial z} = 4(z - 5)^3 - 6(y - z)^5 - 6(x - z)^5 = 0. \quad (8)$$

This produced a system $F(x, y, z) = 0$ with three equations and three unknowns. The initial vector was

$$x_0 = [-1, 1, -1]^T. \quad (9)$$

4.1 Results

The method was run for 10 iterations. The required outputs from selected iterations are shown in Table 1.

Iteration	x	y	z
1	1.1105	2.6810	1.4024
2	2.4736	3.6808	2.8837
8	2.4317	3.0000	3.5683

Table 1: Selected iteration outputs for the nonlinear optimization problem.

4.2 Analysis

This experiment shows how an optimization problem can be solved by converting it into a nonlinear system. The algorithm updated the variables quickly from the initial guess and moved toward a stable region of the solution space.

This part of the project required deriving the gradient equations, building the full Jacobian matrix, and using the third-order Newton-type update correctly. It also showed the connection between optimization and root-finding: instead of minimizing the original function directly, the algorithm solves for the point where the gradient is zero.

5 Experiment 2: Exponential Regression Problem

The second problem involved a nonlinear regression model. The goal was to minimize

$$f(x_1, x_2) = \frac{1}{2} \sum_{j=1}^N [y_j - \exp(x_1 t_j) - \exp(x_2 u_j)]^2. \quad (10)$$

The generated data followed the structure

$$y_j = \exp(\sqrt{3}t_j) + \exp(\sqrt{7}u_j), \quad (11)$$

so the expected solution was

$$(x_1, x_2) = (\sqrt{3}, \sqrt{7}). \quad (12)$$

The initial vector was

$$x_0 = [0.75, 0.25]^T. \quad (13)$$

The system was created by taking the partial derivatives of the objective function with respect to x_1 and x_2 , then setting them equal to zero. Since the objective function includes exponential terms, the resulting equations and Jacobian entries are more sensitive than in the first experiment.

5.1 Results

The method was run for 10 iterations. The selected outputs are shown in Table 2.

Iteration	x_1	x_2
2	-9.2165	-8.3486
5	-23.5653	-18.6448
10	-53.1977	-38.5236

Table 2: Selected iteration outputs for the exponential regression problem.

5.2 Analysis

For this experiment, the method did not converge to the expected solution $(\sqrt{3}, \sqrt{7})$. Instead, the values moved farther away from the solution and became increasingly negative.

This result is important because it shows a practical limitation of Newton-type methods. Even though the method has strong theoretical convergence properties, convergence is not guaranteed from every starting point. The initial vector must be close enough to the solution, and the nonlinear system must behave well around that point.

The exponential regression problem is more sensitive because exponential functions can change very quickly. This can cause the update step to become unstable when the starting point is not in a good region. As a result, the algorithm moved away from the solution instead of toward it.

From a practical perspective, this experiment shows why numerical algorithms should be tested carefully. A successful implementation should not only produce numbers, but also explain whether those numbers make sense.

6 Key Technical Skills Demonstrated

This project demonstrates several skills that are useful in software development, data science, and quantitative computing:

- **Numerical computing:** implemented an iterative method for nonlinear systems.
- **MATLAB programming:** used loops, vectors, matrices, and function evaluations.
- **Optimization:** converted minimization problems into systems of first-order conditions.
- **Linear algebra:** used Jacobian matrices and matrix inverse operations in the update step.
- **Algorithm analysis:** compared convergence and divergence behavior across different problems.
- **Mathematical modeling:** worked with polynomial and exponential models.
- **Technical communication:** explained implementation details and numerical results clearly.

7 Main Takeaways

The main takeaway from this project is that implementing a numerical method requires both coding and analysis. It is not enough to translate formulas into MATLAB; it is also necessary to understand how the algorithm behaves when applied to different systems.

In the first experiment, the method performed well and moved toward a stable solution for a nonlinear optimization problem. In the second experiment, the method failed to converge from the given initial guess, which showed the sensitivity of Newton-type methods to starting values and nonlinear behavior.

Overall, this project strengthened my understanding of numerical optimization, nonlinear systems, Jacobian matrices, and iterative methods. It also gave me experience presenting mathematical work in a way that connects theory, implementation, and practical interpretation.

8 Future Improvements

If this project were extended, the following improvements could be added:

- Compare the third-order Newton-type method directly with the classical Newton method.
- Add convergence plots showing error size across iterations.
- Test different initial guesses for the exponential regression problem.
- Replace explicit matrix inversion with MATLAB's backslash operator for better numerical stability.
- Organize the code into reusable functions instead of keeping all logic inside one script.

9 Conclusion

This project implemented a third-order Newton-type method for solving nonlinear systems and applied it to two different numerical problems. The first problem showed how the method can be used for nonlinear optimization, while the second problem showed that convergence depends strongly on the initial guess and the structure of the model.

The project reflects the full workflow of a numerical computing task: understanding the mathematical method, implementing it in MATLAB, running experiments, analyzing results, and communicating the findings in a clear technical format.

References

- [1] M. T. Darvishi and A. Barati, *A third-order Newton-type method to solve systems of nonlinear equations*, Applied Mathematics and Computation, 187(2), 630–635, 2007.